

BCL1223 — DATABASE FUNDAMENTALS

Final Assessment Practice Exam & Model Answer Scheme

Student Name: Chan Jing Yi

Student ID: SUOL2500321

Course Code: BCL1223

Course Title: Database Fundamentals

Assessment Mode: Closed-Book Proctored Mock Exam

Date: August 6, 2026

Executive Summary & Exam Guidelines

This practice examination document has been compiled to prepare for the **BCL1223 Database Fundamentals Final Assessment**. The assessment tests key theoretical concepts, conceptual database design, normalization techniques, and practical SQL query formulation.

The exam paper is structured into four main technical sections matching the course syllabus and live assessment requirements:

- Section A: Core Database Concepts & Relational Architecture** (20 Marks)
- Section B: Entity-Relationship Diagram (ERD) & Conceptual Data Modeling** (25 Marks)
- Section C: Database Normalization (UNF to 3NF)** (25 Marks)
- Section D: SQL Query Writing & Syntax Implementation** (30 Marks)

SECTION A: Core Database Concepts & Relational Architecture (20 Marks)

Question 1.1: Database Systems vs. Traditional File Processing

Compare traditional file-processing systems with modern Relational Database Management Systems (RDBMS). Explain how physical and logical data independence reduce system maintenance overhead. (4 Marks)

Model Answer:

Traditional file-processing systems store data in isolated files maintained by separate application programs. This architecture introduces severe data redundancy, structural dependence, data inconsistency, and difficult ad-hoc reporting. Conversely, a Relational Database Management System (RDBMS) centralizes data storage and decouples data access from application logic through the ANSI-SPARC three-schema architecture.

- Logical Data Independence:** The immunity of external schemas and application programs to changes in the conceptual schema (e.g., adding a new attribute or table). Applications remain unaffected unless they directly interact with altered structures.
- Physical Data Independence:** The immunity of the conceptual schema to changes in physical storage structures (e.g., changing file indexing schemes, moving disk volumes, or altering B-tree structures). The physical layout can be optimized without modifying logical queries or software interfaces.

Question 1.2: Integrity Constraints Analysis

Distinguish between **Entity Integrity** and **Referential Integrity**. Provide a concrete scenario demonstrating how violating referential integrity damages database consistency. (4 Marks)

Model Answer:

- Entity Integrity:** Enforces that no primary key attribute can be null (`NULL`). Because primary keys serve as unique row identifiers, a null primary key value prevents uniquely identifying tuple instances within a relation.
- Referential Integrity:** Enforces that a foreign key value in a child relation must either match a valid candidate key value in a referenced parent relation or be entirely null (if optional).

Violation Scenario: Consider a `STUDENT` table (parent) and a `CLUB_MEMBERSHIP` table (child). If a user attempts to insert a record into `CLUB_MEMBERSHIP` with `Student_ID = 'S999'` when student 'S999' does not exist in `STUDENT`, referential integrity is violated. Allowing this orphan record creates an invalid reference, leading to failed join operations, erroneous report aggregations, and corrupted database integrity.

Question 1.3: Relational Keys Classification

Define and contrast the following database keys: **Candidate Key**, **Primary Key**, **Composite Key**, and **Foreign Key**. (4 Marks)

Model Answer:

1. **Candidate Key:** A minimal superkey—a minimal set of attributes that uniquely identifies each tuple in a relation without any redundant attributes.
2. **Primary Key:** The specific candidate key chosen by the database designer to serve as the principal tuple identifier for a relation.
3. **Composite Key:** A primary or candidate key that consists of two or more attributes combined (e.g., {Student_ID, Club_ID} in a membership relation).
4. **Foreign Key:** An attribute or set of attributes in one relation that references the primary key of another relation, establishing a logical linkage between tables.

Question 1.4: Relational Algebra Operations

Given the relational schema STUDENT(StudentID, Name, Faculty, GPA) and ENROLLMENT(StudentID, CourseID, Grade), express the following query in formal Relational Algebra: "Retrieve the Name and GPA of all students in the 'FICT' faculty who achieved a grade of 'A' in course 'BCL1223'." (4 Marks)

Model Answer:

To express this query, we apply selection (σ), natural join ($\bowtie$), and projection (π):

$$\pi_{\text{Name,GPA}} (\sigma_{\text{Faculty}='FICT'}(\text{STUDENT}) \bowtie \sigma_{\text{CourseID}='BCL1223' \wedge \text{Grade}='A'}(\text{ENROLLMENT}))$$

Question 1.5: DBMS Architecture

Explain the three levels of the ANSI-SPARC Architecture and the role of the Database Administrator (DBA) in managing schema mappings. (4 Marks)

Model Answer:

1. **External Level (User Views):** Tailored perspectives of the database presented to specific end-user groups.
2. **Conceptual Level (Community View):** The complete logical structure of the entire database, describing all entities, relationships, data types, and integrity constraints.
3. **Internal Level (Physical View):** The low-level representation detailing physical storage allocation, access paths, indexing, and data compression techniques.

SECTION B: Entity-Relationship Diagram (ERD) & Conceptual Data Modeling (25 Marks)

Scenario Description: SEGi University Student Affairs Department requires an interactive database system to manage student clubs, campus events, venue bookings, and academic advisors.

Business Rules:

- A **Student** can join multiple **Clubs**, and a **Club** consists of multiple **Students**.
- Each **Club** must be advised by exactly one **Advisor** (Lecturer), but an **Advisor** can advise one or more **Clubs**.
- Each **Club** organizes one or more **Events**. Each **Event** is organized by exactly one **Club**.
- Each **Event** is held at exactly one **Venue**, while a **Venue** can host multiple **Events**.
- A **Staff Member** is assigned to manage one or more **Venues**, but each **Venue** is managed by exactly one designated **Staff Member**.

Question 2.1: Structural Entity & Multiplicity Analysis

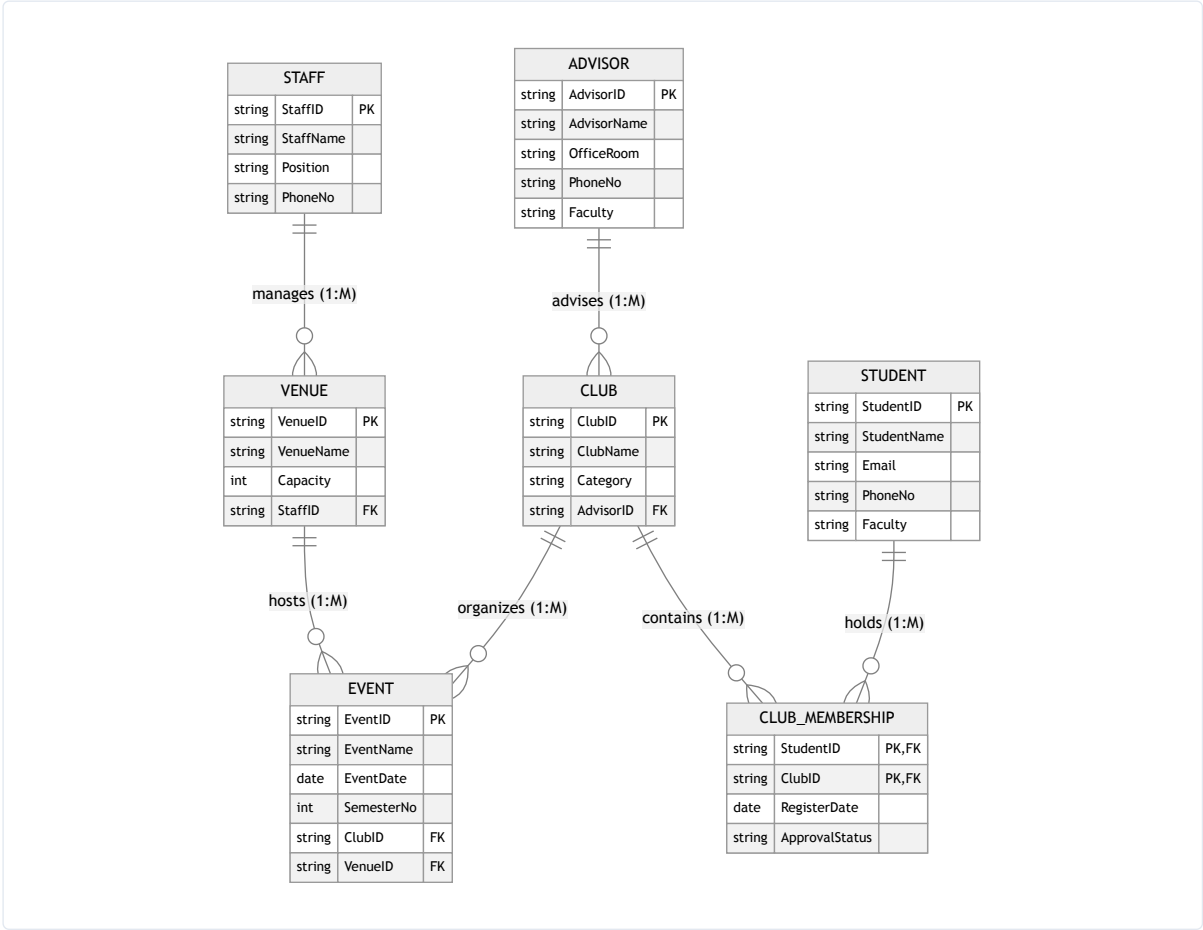
Identify all entities, primary keys, foreign keys, and cardinalities. Explain how the Many-to-Many (\$M:N\$) relationship between `STUDENT` and `CLUB` is resolved. (8 Marks)

Model Answer:

- **Entities & Keys:** `STUDENT` (PK: `StudentID`), `CLUB` (PK: `ClubID`, FK: `AdvisorID`), `ADVISOR` (PK: `AdvisorID`), `EVENT` (PK: `EventID`, FK: `ClubID`, `VenueID`), `VENUE` (PK: `VenueID`, FK: `StaffID`), `STAFF` (PK: `StaffID`), `CLUB_MEMBERSHIP` (Junction Entity, PK: {`StudentID`, `ClubID`}).
- **Multiplicities:** `Advisor:Club` (1:M), `Club:Event` (1:M), `Venue:Event` (1:M), `Staff:Venue` (1:M), `Student:Club` (M:N decomposed via `CLUB_MEMBERSHIP`).

Question 2.2: Mermaid ERD Diagram Generation

Construct a complete Entity-Relationship Diagram (ERD) using Mermaid.js syntax reflecting all entities, primary keys, foreign keys, and cardinalities. (10 Marks)



Question 2.3: Data Dictionary Construction

Formulate a Data Dictionary table for the CLUB_MEMBERSHIP and EVENT entities. (7 Marks)

Data Dictionary: CLUB_MEMBERSHIP Entity

Attribute Name	Data Type	Field Size	Key Type	Nullability	Constraint Description
StudentID	VARCHAR2	10	PK, FK	NOT NULL	References STUDENT (StudentID)
ClubID	VARCHAR2	10	PK, FK	NOT NULL	References CLUB (ClubID)
RegisterDate	DATE	Default	None	NOT NULL	System registration date
ApprovalStatus	VARCHAR2	15	None	NOT NULL	CHECK (ApprovalStatus IN ('APPROVED', 'PENDING', 'REJECTED'))

SECTION C: Database Normalization (UNF to 3NF) (25 Marks)

Scenario: Convert an unnormalized event register report into 3NF.

Question 3.1: Unnormalized Form (UNF)

UNF: EVENT_REGISTER(InvoiceNo, InvoiceDate, StudentID, StudentName, StudentEmail, Faculty, {EventID, EventName, EventDate, ClubID

Question 3.2: First Normal Form (1NF) & Functional Dependencies

1NF Schema: EVENT_REGISTER_1NF(InvoiceNo, EventID, InvoiceDate, StudentID, StudentName, StudentEmail, Faculty, EventName, EventDate, ClubID, ClubName, AdvisorID, AdvisorName)

Functional Dependencies (FDs):

- FD1: {InvoiceNo, EventID} → Fee (Full FD)
- FD2: InvoiceNo → InvoiceDate, StudentID, StudentName, StudentEmail, Faculty (Partial FD)
- FD3: StudentID → StudentName, StudentEmail, Faculty (Transitive FD)
- FD4: EventID → EventName, EventDate, ClubID, ClubName, AdvisorID, AdvisorName (Partial FD)
- FD5: ClubID → ClubName, AdvisorID, AdvisorName (Transitive FD)
- FD6: AdvisorID → AdvisorName (Transitive FD)

Question 3.3 & 3.4: 2NF & Final 3NF Normalization Schema

Final 3NF Relations:

- STUDENT(StudentID, StudentName, StudentEmail, Faculty)
- INVOICE(InvoiceNo, InvoiceDate, StudentID^*)
- ADVISOR(AdvisorID, AdvisorName)
- CLUB(ClubID, ClubName, AdvisorID^*)
- EVENT(EventID, EventName, EventDate, ClubID^*)
- INVOICE_LINE(InvoiceNo^*, EventID^*, Fee)

SECTION D: SQL Query Writing & Syntax Implementation (30 Marks)

Question 4.1: DDL Table Creation Script

```
-- 1. Create ADVISOR Table
CREATE TABLE ADVISOR (
    AdvisorID    VARCHAR2(10) NOT NULL,
    AdvisorName  VARCHAR2(100) NOT NULL,
    OfficeRoom   VARCHAR2(20),
    PhoneNo      VARCHAR2(15),
    Faculty      VARCHAR2(50) NOT NULL,
    CONSTRAINT pk_advisor PRIMARY KEY (AdvisorID)
);

-- 2. Create CLUB Table
CREATE TABLE CLUB (
    ClubID       VARCHAR2(10) NOT NULL,
    ClubName     VARCHAR2(100) NOT NULL,
    Category     VARCHAR2(50) NOT NULL,
    AdvisorID    VARCHAR2(10) NOT NULL,
    CONSTRAINT pk_club PRIMARY KEY (ClubID),
    CONSTRAINT uq_club_name UNIQUE (ClubName),
    CONSTRAINT fk_club_advisor FOREIGN KEY (AdvisorID)
        REFERENCES ADVISOR(AdvisorID) ON DELETE RESTRICT
);

-- 3. Create CLUB_MEMBERSHIP Table
CREATE TABLE CLUB_MEMBERSHIP (
    StudentID    VARCHAR2(10) NOT NULL,
    ClubID       VARCHAR2(10) NOT NULL,
    RegisterDate DATE DEFAULT SYSDATE NOT NULL,
    ApprovalStatus VARCHAR2(15) DEFAULT 'PENDING' NOT NULL,
    CONSTRAINT pk_club_membership PRIMARY KEY (StudentID, ClubID),
    CONSTRAINT chk_approval_status
        CHECK (ApprovalStatus IN ('APPROVED', 'PENDING', 'REJECTED'))
);
```

Question 4.2 to 4.5: Advanced DML Queries

Query: Aggregating Advisors managing >1 Club

```
SELECT
    a.AdvisorID,
    a.AdvisorName,
    COUNT(c.ClubID) AS TotalClubsAdvised
FROM ADVISOR a
INNER JOIN CLUB c ON a.AdvisorID = c.AdvisorID
GROUP BY a.AdvisorID, a.AdvisorName
HAVING COUNT(c.ClubID) > 1
ORDER BY TotalClubsAdvised DESC;
```

Query: Conditional Pivot Aggregation (Semester Event Counts)

```
SELECT
    a.AdvisorID,
    a.AdvisorName,
    SUM(CASE WHEN e.SemesterNo = 1 THEN 1 ELSE 0 END) AS Sem1_Event_Count,
    SUM(CASE WHEN e.SemesterNo = 2 THEN 1 ELSE 0 END) AS Sem2_Event_Count,
    COUNT(e.EventID) AS Total_Events_Organized
FROM ADVISOR a
INNER JOIN CLUB c ON a.AdvisorID = c.AdvisorID
INNER JOIN EVENT e ON c.ClubID = e.ClubID
GROUP BY a.AdvisorID, a.AdvisorName
ORDER BY Total_Events_Organized DESC;
```